

INSIGHT

AI-Powered Interview Practice Partner

Architecture & Design Analysis

4

AI AGENTS

~4K

LINES

12

ROLES

5

PHASES

9

ENDPOINTS

ABSTRACT

This report presents a comprehensive architectural and technical analysis of Insight, a multi-agent AI interview simulation platform. The system orchestrates four specialized LLM agents — a behavioral profiler, a scoring grader, a dynamic interviewer, and a feedback synthesizer — over a real-time WebSocket/SSE communication layer, backed by Redis session memory and ChromaDB vector retrieval. Covers system design rationale, data-flow topology, technology selection justifications, security architecture, and scalability.



TABLE OF CONTENTS



§	Section	Contents
§ 01	System Overview & Vision	Problem statement, stakeholders, innovations
§ 02	High-Level Architecture	Three-tier system topology, all components
§ 03	Multi-Agent System Design	Agent roles, pipeline, interaction sequence
§ 04	Technology Stack & Rationale	All technologies with alternatives analysis
§ 05	Repository Structure	File-by-file component breakdown
§ 06	API Endpoints & Protocols	REST, WebSocket, SSE endpoint reference
§ 07	Data Flow & Session State	Lifecycle, state schema, Redis design
§ 08	Interview Phase Engine	State machine, persona archetypes, RAG
§ 09	Evaluation & Scoring Model	Weighted rubric, dimensions, trend analysis
§ 10	Frontend Architecture	SPA components, coding UI, voice, analytics
§ 11	Security Architecture	Sandbox, isolation, controls matrix
§ 12	Scalability & Deployment	Horizontal scaling, Docker, configuration
§ 13	Code Quality & Roadmap	Assessment, future development priorities

01 SYSTEM OVERVIEW & VISION

Insight is a role-based AI interview simulation platform that provides candidates with realistic, adaptive, and analytically rich mock interview experiences without requiring human interviewers. It addresses a critical accessibility gap: high-quality interview coaching has historically required costly expert consultants inaccessible to most candidates globally.

The platform ingests a candidate's resume (PDF) and a job description, then orchestrates a structured interview through four specialized LLM agents — each with isolated responsibility mirroring how real-world assessment panels divide evaluation labor for greater analytical depth.

PROBLEM STATEMENT

Dimension	Problem Being Solved	Impact
Accessibility	High-quality coaching locked behind expensive consultants	■ High
Consistency	Human interviewers deliver inconsistent quality and varying biases	■ High
Availability	Scheduling constraints limit practice frequency to 1–2x per week	■ Medium
Feedback Depth	Most tools give surface-level feedback with no behavioral analysis	■ High
Adaptivity	Static question banks do not adapt to live candidate responses	■ Medium
Code Practice	Coding challenges require complex sandboxed infrastructure	■ Medium

KEY INNOVATIONS

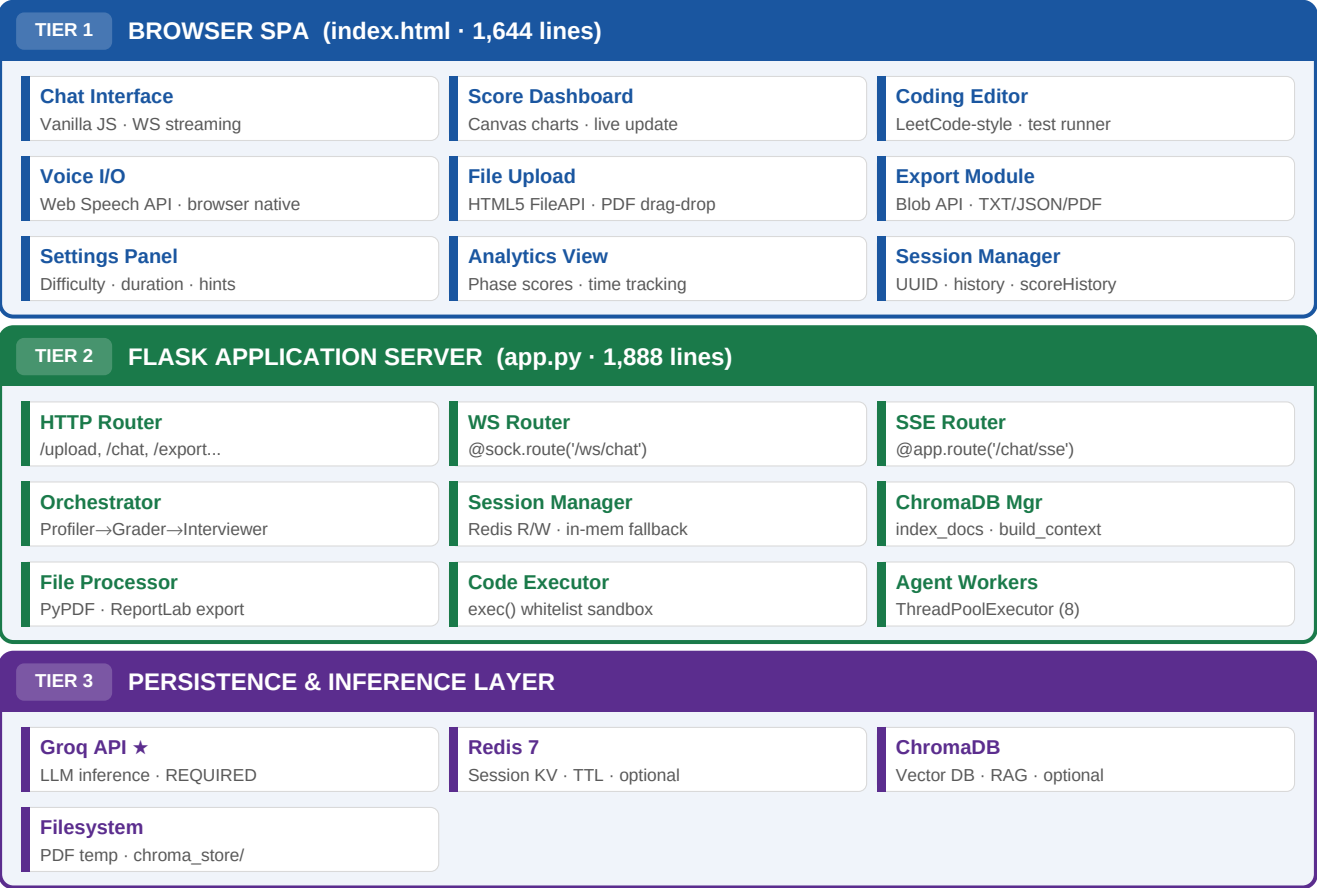
Innovation	Technical Detail
◆ Multi-Agent Architecture	Four independent agents each with their own system prompt, evaluation schema, and JSON output contract. Orchestrator sequences them per turn — enabling richer analysis than any monolithic prompt.
■ Streaming Real-Time UX	Groq sub-200ms inference + WebSocket/SSE token streaming creates a natural conversational feel with no perceptible latency between turns.
■ Resume-Aware RAG	ChromaDB indexes chunks of the resume and JD at session start. Semantic retrieval injects the most relevant context into each Interviewer call — making questions genuinely personalized.
■ Adaptive Difficulty	Profiler detects behavioral persona each turn (8 archetypes). Interviewer dynamically adjusts tone, depth, and follow-up strategy based on live behavioral signals.
■ Safe Code Execution	Python exec() with whitelist-only builtins. AST sanitizer strips dangerous patterns. No external sandboxing service required.
■ Graceful Degradation	Redis and ChromaDB are both optional. System falls back to in-memory state — zero required infrastructure beyond a Groq API key.

SUPPORTED ROLES (12 PROFILES)

Role	Focus Areas	Role	Focus Areas
Software Engineer	DSA, System Design, OOP	Data Scientist	ML, Statistics, Model Eval
ML Engineer	MLOps, Deployment, Training	Product Manager	Strategy, Metrics, Stakeholders
DevOps Engineer	CI/CD, Infrastructure, SRE	Data Engineer	Pipelines, ETL, Big Data
Frontend Engineer	UI/UX, Frameworks, Perf	Backend Engineer	APIs, Databases, Scalability
QA Engineer	Testing, Quality, Automation	Security Engineer	Threat Modeling, Protocols
Cloud Architect	Cloud Platforms, Patterns	Sales Engineer	Technical Sales, Demos

02 HIGH-LEVEL ARCHITECTURE

The system is organized into three principal tiers with clear separation of concerns. The browser tier handles presentation and real-time communication. The application tier orchestrates the multi-agent pipeline and file processing. The persistence/inference tier provides durable state management, semantic retrieval, and LLM inference.



◆ **Architecture Principle:**
The application server is fully stateless per-request — all mutable state lives exclusively in Redis (or in-memory dict). Multiple Flask instances can serve any request interchangeably, enabling horizontal scaling with load balancer + WebSocket affinity

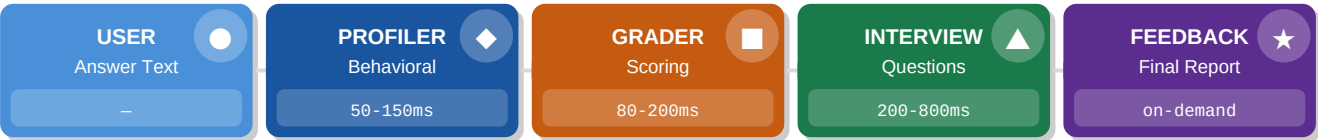
TIER COMPARISON

Attribute	Tier 1 — Browser	Tier 2 — App Server	Tier 3 — Persistence
Primary Tech	Vanilla JS + Tailwind	Python + Flask	Redis + ChromaDB + Groq
State Held	JS variables (ephemeral)	None (stateless)	All durable session state
Scaling	Scales with browsers	Horizontal (add instances)	Redis cluster + Chroma vol
Failure Mode	Reconnects via WS retry	Stateless — any restart ok	Fallbacks to in-memory
Communication	WebSocket + SSE + HTTP	HTTP + WS + SSE + Groq API	Redis protocol + HTTP API

03 MULTI-AGENT SYSTEM DESIGN

Rather than a single monolithic LLM prompt that must simultaneously conduct an interview, score responses, detect behavioral patterns, and generate feedback — Insight decomposes these concerns into four isolated, specialized agents each with their own system prompt, evaluation schema, and strict JSON output contract.

AGENT PIPELINE DIAGRAM



Agent pipeline — left to right per user turn. Feedback Agent triggered on session end only.

PER-TURN INTERACTION SEQUENCE

Step	Actor	Input	Output	Latency
1	User	Keyboard / Voice	Answer text	<5ms
2	Orchestrator	Answer + session_id	Loaded state dict	<10ms
3	Profiler Agent	Answer + history	JSON: persona + flags	50–150ms
4	Grader Agent	Answer + Q + JD + scores	JSON: score + feedback	80–200ms
5	ChromaDB	Current question text	Top-4 context chunks	5–20ms
6	Interviewer	All above merged	Token stream → WS/SSE	200–800ms
7	Orchestrator	Stream complete	Updated state → Redis	<10ms
8	Frontend	WS/SSE token frames	Rendered message + chart	Real-time

AGENT SPECIFICATIONS

AGENT 01 ProfilerAgent profiler.py · 244 lines	
Primary Method	<code>analyze(user_input: str, context_history: list) → dict</code>
Purpose	Behavioral pattern analysis. Detects persona archetype (8 types), sentiment polarity, and red flags including memorization, knowledge gaps, and inconsistency across turns.
Key Fields	Personas: <code>confused · efficient · chatty · edge_case · normal · silent · anxious · overconfident</code> Sentiment: <code>positive · neutral · negative</code> Confidence: <code>float 0.0–1.0</code>
Output Contract	<code>{ persona, is_relevant, advice, confidence, sentiment, red_flags[], memorization_detected, knowledge_gaps[], inconsistency_detected }</code>

AGENT 02	GraderAgent	grader.py · 318 lines
Primary Method	<code>evaluate(user_input, current_question, jd_text, resume_text, previous_scores) → dict</code>	
Purpose	4-dimension weighted scoring: Technical Accuracy (40%), Communication (25%), Problem Solving (20%), Relevance (15%). Maintains trend analysis across session. Seniority calibration for senior roles.	
Key Fields	Score bands: 90–100 Flawless · 70–89 Good · 50–69 Adequate · 30–49 Weak · 0–29 Poor Trend: improving / stable / declining Follow-up trigger: technical < 60 or problem_solving < 55	
Output Contract	<code>{ score, technical_accuracy, communication, problem_solving, relevance, strengths[], weaknesses[], improvements[], requires_followup, detailed_feedback }</code>	

AGENT 03	InterviewerAgent	interviewer.py · 453 lines
Primary Method	<code>generate_response_stream(user_input, history, resume, jd, profiler_data, grader_data, ...) → Generator[str]</code>	
Purpose	Conversational core. Persona: 'Byte' — elite senior technical recruiter with 20+ years experience. Streams tokens via WebSocket/SSE. Adapts tone, depth, and follow-up strategy based on profiler persona and grader scores.	
Key Fields	Context sources: Resume + JD + ChromaDB top-4 chunks + full history + profiler JSON + grader JSON Adaptive signals: overconfident→deeper probing · anxious→reassuring · confused→break down · efficient→probe deeper	
Output Contract	<code>Raw token stream (not JSON) – displayed progressively via WebSocket/SSE in the frontend chat UI</code>	

AGENT 04	FeedbackGeneratorAgent	feedback_generator.py · 158 lines
Primary Method	<code>generate_comprehensive_feedback(interview_history, resume, jd, all_scores, question_count) → str</code>	
Purpose	Post-session report generator. Invoked on-demand at session end only, not per-turn. Produces a 7-section structured Markdown report synthesizing the full interview transcript, scores, and JD alignment.	
Key Fields	Sections: 1. Executive Summary 2. Performance Breakdown 3. Strengths (with examples) 4. Improvement Areas 5. STAR Method Evaluation 6. Technical Competency Map 7. Prioritized Action Roadmap	
Output Contract	<code>Markdown-formatted string → rendered in frontend panel or exported as PDF via ReportLab</code>	

04

TECHNOLOGY STACK & RATIONALE



Every technology was selected against explicit criteria. For each, a viable alternative was considered and rejected with documented reasoning — serving as the project’s architectural decision record.

BACKEND TECHNOLOGIES

Technology	Version	Role	Why Selected	Rejected Alternative
Python	3.9+	Language	Native LLM SDK; ML ecosystem; async support	Node.js — weaker LLM SDK maturity
Flask	3.x	Web Framework	Minimal structure; no forced ORM or patterns	Django — batteries-included overhead
flask-sock	0.7+	WebSocket	Clean @sock.route decorator API for WS	Socket.IO — requires matching client lib
Groq SDK	0.9+	LLM Client	Sub-200ms TTFT; streaming; model variety	OpenAI SDK — higher latency + cost
Redis	7.x	Session Store	In-memory KV; O(1) ops; TTL auto-expiry	PostgreSQL — overkill for ephemeral state
ChromaDB	0.5+	Vector DB	Local-first; Python-native; no API key	Pinecone — cloud account + egress cost
PyPDF	3.x	PDF Parse	Pure Python; no binary dependencies	pdfminer.six — more boilerplate API
ReportLab	4.x	PDF Export	Programmatic PDF; tables; headers/footers	WeasyPrint — HTML intermediate needed
python-dotenv	1.x	Config	Industry-standard .env; env promotion	Hardcoded config — security violation

FRONTEND TECHNOLOGIES

Technology	Category	Why Selected	Rejected Alternative
Vanilla JS ES6+	Language	No framework overhead; direct WebSocket API control; faster load	React — reconciliation hurts streaming UX
Tailwind CSS	Styling	Utility-first; no custom CSS arch; CDN delivery; consistent	Custom CSS — verbose without design system
Web Speech API	Voice I/O	Native browser; zero cost; fully private — audio stays local	Deepgram — API cost + audio data exposure
WebSocket API	Realtime	Built-in bidirectional streaming; no additional library	Socket.IO client — unnecessary abstraction

Technology	Category	Why Selected	Rejected Alternative
Canvas API	Charts	Lightweight real-time charts without pulling in Chart.js	Chart.js — heavy for simple line charts
Blob API	Export	Client-side file downloads without server round-trip	Server download — unnecessary request

05

REPOSITORY STRUCTURE

File / Directory	Size	Type	Key Contents & Responsibilities
app.py	1,888 lines	Python	Flask server · all routes · orchestration · session mgmt · ChromaDB ops · code sandbox · export handlers
agents/profiler.py	244 lines	Python	ProfilerAgent class · behavioral analysis system prompt · persona detection · red flag pattern matching
agents/grader.py	318 lines	Python	GraderAgent class · 4-dimension scoring rubric · trend analysis · seniority calibration · feedback gen
agents/interviewer.py	453 lines	Python	InterviewerAgent class · 'Byte' persona · phase management · adaptive questioning · 12 role profiles
agents/feedback_gen.py	158 lines	Python	FeedbackGeneratorAgent · 7-section report structure · STAR eval · competency map · roadmap synthesis
templates/index.html	1,644 lines	HTML/JS	Complete SPA: chat UI · score dashboard · coding editor · voice controls · export panel · analytics
requirements.txt	~12 lines	Config	Pinned: flask · flask-cors · flask-sock · groq · python-dotenv · pypdf · reportlab · redis · chromadb
.env	~5 lines	Secret	GROQ_API_KEY · REDIS_URL · SESSION_TTL_SECONDS · CHROMA_DIR · AGENT_WORKERS (never commit to git)
chroma_store/	Directory	Binary	ChromaDB persistence · session-namespaced embedding collections · excluded from git via .gitignore



Total Core Code:

~4,700 lines across 6 source files. The project is remarkably lean for a full multi-agent interview platform with real-time streaming,

06

API ENDPOINTS & COMMUNICATION PROTOCOLS



Endpoint	Method	Protocol	Input	Response / Behavior
/	GET	HTTP	—	Serves index.html single-page application
/upload-context	POST	HTTP	multipart: resume, jd, role	Extracts PDF · indexes ChromaDB · returns { session_id }
/chat	POST	HTTP	JSON: { message, session_id }	Full agent pipeline · returns complete JSON response
/chat/sse	GET	SSE	URL params: session_id, message	Server-sent events · each data: chunk = one Interviewer token
/ws/chat	—	WS	JSON frames: { session_id, message }	Bidirectional WS · runs pipeline · streams tokens back
/run-tests	POST	HTTP	JSON: { code, language, session_id }	Sandbox exec · returns { passed, failed, results[] }
/end-interview	POST	HTTP	JSON: { session_id }	Triggers FeedbackGeneratorAgent · returns Markdown report
/export-transcript	GET	HTTP	URL: session_id, format (txt json pdf)	Streams downloadable transcript in requested format
/session-state	GET	HTTP	URL param: session_id	Returns current session state JSON for frontend sync



■ Common 422 Cause:

/upload-context MUST use multipart/form-data (not application/json) because it receives a binary PDF file. Sending JSON to this endpoint is the most frequent source of 422 Unprocessable Entity errors. All other endpoints expect application/json.

SESSION LIFECYCLE — 13 STEPS

Step	Phase	Actions Performed	Storage
1	Init	UUID session_id generated · default_session_state() created with all fields at null/defaults	Redis/mem
2	Upload	PDF parsed by PyPDF · JD stored · role set · text chunked and embedded into ChromaDB session collection	Redis + Chroma
3	Start	started=True written · Q0 generated by Interviewer · streamed to client via WebSocket	Redis
4	Load	User submits answer · orchestrator loads full session state from Redis · read-only load	Redis (read)
5	RAG	build_retrieval_context() queries ChromaDB with current question · top-4 chunks returned and merged	Redis cache
6	Profiler	ProfilerAgent.analyze() → Groq API call → JSON: persona, sentiment, red_flags, confidence	In-memory
7	Grader	GraderAgent.evaluate() → Groq API call → composite score + 4-dimension breakdown + feedback	In-memory
8	Stream	InterviewerAgent streams tokens via Groq → forwarded to client over WebSocket in real-time	Browser DOM
9	Persist	Score→all_scores[] · Q&A;→qa_log[] · messages→history[] · flags merged · count++ · phase evaluated	Redis (write)
10	Index	add_answer_to_index() adds candidate answer to ChromaDB collection for future cross-turn retrieval	ChromaDB
11	Repeat	Steps 4–10 repeat for each subsequent turn until question_count ≥ 10 or session manually ended	Redis+Chroma
12	Feedback	FeedbackGeneratorAgent synthesizes full history → 7-section Markdown report generated	Redis
13	Cleanup	Export file streamed to browser · Redis TTL auto-expires session after SESSION_TTL_SECONDS	FS → TTL del

SESSION STATE SCHEMA (15 FIELDS)

Field	Type	Default	Purpose & Consumers
session_id	str	UUID4	Redis key prefix · ChromaDB collection name · prevents enumeration
resume	str	''	Full PDF text · passed to GraderAgent + InterviewerAgent every turn

Field	Type	Default	Purpose & Consumers
<code>jd</code>	<code>str</code>	<code>"</code>	Job description text · drives scoring relevance + question domain focus
<code>selected_role</code>	<code>str</code>	<code>"</code>	Maps to InterviewerAgent <code>role_info</code> dict with question domains
<code>interview_phase</code>	<code>str</code>	<code>'intro'</code>	Phase enum: <code>intro</code> / <code>technical</code> / <code>behavioral</code> / <code>deep_dive</code> / <code>feedback</code>
<code>question_count</code>	<code>int</code>	<code>0</code>	Drives phase transitions · <code>0=intro</code> , <code>1–3=technical</code> , <code>4–6=behavioral...</code>
<code>current_question</code>	<code>str</code>	<code>"</code>	Last Q posed · used by GraderAgent as context for scoring the answer
<code>started</code>	<code>bool</code>	<code>False</code>	True after <code>/upload-context</code> and first Interviewer turn complete
<code>all_scores</code>	<code>list[int]</code>	<code>[]</code>	Per-turn composite scores · trend analysis · frontend chart data
<code>interview_history</code>	<code>list[dict]</code>	<code>[]</code>	Full <code>[[{role, content}]]</code> conversation · passed to Interviewer as context window
<code>qa_log</code>	<code>list[dict]</code>	<code>[]</code>	<code>{question, answer, score, feedback}</code> pairs · FeedbackGenerator source data
<code>edge_cases_detected</code>	<code>list[str]</code>	<code>[]</code>	Off-topic or jailbreak inputs flagged by Profiler · logged for report
<code>red_flags_history</code>	<code>list[str]</code>	<code>[]</code>	Behavioral flags accumulated: <code>memorization</code> · <code>gaps</code> · <code>inconsistency</code>
<code>coding_round</code>	<code>dict None</code>	<code>None</code>	Active coding challenge: <code>{problem, test_cases, language, starter_code}</code>
<code>retrieval_cache</code>	<code>dict</code>	<code>{}</code>	<code>{query → chunks}</code> · prevents redundant ChromaDB calls within a session

08 INTERVIEW PHASE ENGINE

The Interviewer Agent is governed by a five-phase state machine. Phase transitions are triggered by question_count thresholds, ensuring structured progression from rapport-building through technical depth to behavioral exploration and finally feedback synthesis.

PHASE STATE MACHINE



Phase	Name	Q Range	Trigger	Interviewer Focus & Behavior
0	Introduction	Q0	count == 0	Warm welcome · establish rapport · ask candidate to introduce themselves and walk through their background
1	Technical	Q1–Q3	$1 \leq n \leq 3$	Domain-specific technical questions aligned to role profile · Grader applies full 4-dimension rubric · high depth
2	Behavioral	Q4–Q6	$4 \leq n \leq 6$	STAR-method behavioral questions: leadership · conflict · failure · collaboration · FeedbackGen evaluates STAR quality
3	Deep Dive	Q7–Q9	$7 \leq n \leq 9$	Synthesizes signals from prior turns · probes weak areas flagged by Grader · explores inconsistencies · max difficulty
4	Feedback	Q10+	$n \geq 10$ or end	Session conclusion · verbal summary · FeedbackGeneratorAgent triggered · export options presented to candidate

PERSONA ARCHETYPES & ADAPTIVE RESPONSES

Persona	Detection Signals	Interviewer Adaptation	Example Response Change
normal	Professional, engaged, relevant, appropriate length	Maintain standard pace and depth	Follow phase plan as designed — no change
confused	Vague answers, hedging language, incomplete sentences	Break into smaller parts; framing hints	"Start with just the data structure — which would you choose?"
efficient	Brief, correct, minimal elaboration, finishes quickly	Probe with why/how/trade-offs questions	"That's correct — walk me through your reasoning."
chatty	Off-topic stories, personal anecdotes, high word count	Gentle redirect back to technical question	"Interesting — let's bring it back to the technical decision."
anxious	Self-doubt phrases, over-apologizing, excessive hedging	Reassure; rephrase if needed; no reduced difficulty	"You're on the right track — take your time."

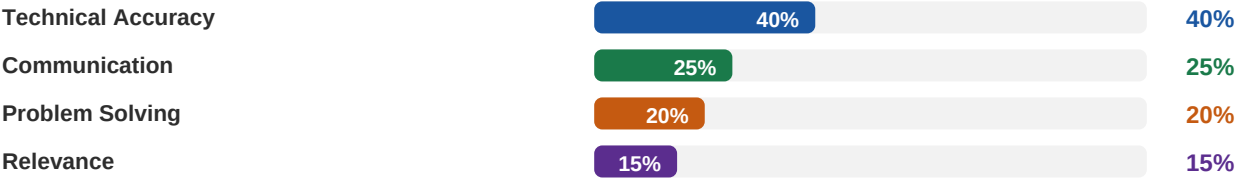
Persona	Detection Signals	Interviewer Adaptation	Example Response Change
overconfident	Dismissive, skips reasoning, assumes expertise obvious	Stress test: edge cases, failure modes, 10x scale	"Good high-level answer — how does this behave under 10x traffic?"
edge_case	Off-topic, jailbreak attempts, system-breaking inputs	Politely decline; log; redirect to interview	"Let's stay focused on your technical interview today."
silent	No response or [SYSTEM_TIMEOUT] signal detected	Offer to rephrase or approach differently	"Would you like me to approach this from a different angle?"

09

EVALUATION & SCORING MODEL



SCORING DIMENSION VISUALIZATION



Weighted scoring dimensions — bars proportional to weight contribution to composite score.

FOUR-DIMENSION WEIGHTED RUBRIC

Dimension	Weight	Evaluated Criteria	Rubric Bands (all dims 0–100)
Technical Accuracy	40%	Factual correctness · proper terminology · no misconceptions · depth relative to role seniority	90–100: Flawless · 70–89: Good, minor errors · 50–69: Adequate, notable issues · 30–49: Weak, major gaps · 0–29: Fundamentally wrong
Communication	25%	Clarity · logical flow · structure · professional tone · concise without losing depth	90–100: Exceptionally clear · 70–89: Clear, minor awkwardness · 50–69: Understandable, disorganized · 0–49: Confusing or incoherent
Problem Solving	20%	Methodology · edge-case consideration · trade-off reasoning · creative alternatives · systematic approach	90–100: Systematic + edge cases · 70–89: Good approach, misses some · 50–69: Correct intuition, shallow · 0–29: No methodology
Relevance	15%	Stays on-topic · directly addresses question · aligned to JD requirements · avoids tangents	90–100: Perfectly on-topic, JD-aligned · 70–89: Mostly relevant · 50–69: Significant drift · 0–29: Off-topic or non-responsive



Composite Formula:

score = (technical × 0.40) + (communication × 0.25) + (problem_solving × 0.20) + (relevance × 0.15) — All sub-dimensions scored 0–100 independently by the LLM before weighted combination. Trend analysis classifies trajectory as improving / stable / declining

TREND ANALYSIS & CALIBRATION FEATURES

Feature	Implementation Detail
Trend Detection	GraderAgent receives previous_scores list each turn. Rising (+5/turn) = improving · flat (±5) = stable · falling = declining
Trend-Aware Feedback	Improving trend: raises bar for next question · Declining trend: more supportive feedback; Interviewer may simplify approach slightly
Seniority Calibration	Senior roles (Staff, Principal, Architect) apply +10-point rubric floor — answers scoring 70 for mid-level may score 55 at staff level

Feature	Implementation Detail
Follow-up Trigger	requires_followup=True when: technical_accuracy < 60 OR problem_solving < 55 · Signals Interviewer to probe the same topic deeper
Score Persistence	All composite scores appended to all_scores[] in Redis each turn · enables chart rendering, final average, and FeedbackGenerator narrative

The frontend is a 1,644-line single-page application built with vanilla JavaScript ES6+ and Tailwind CSS. The choice of vanilla JS over React or Vue was deliberate: direct WebSocket API control and no reconciliation overhead are critical for the real-time streaming interview UX.

SPA COMPONENT BREAKDOWN

Component	JS Pattern	State Managed	Key Behavior
Chat Container	Event-driven DOM	history[], WS conn obj, stream buffer	Receives WS token frames · appends to buffer · renders complete message on stream-end
Score Tracker	Closure + Array	scoreHistory[], trend label, phase ctrs	Canvas line chart redrawn on each score update · per-phase color coding · 10-turn window
Session Manager	Module pattern	currentSessionId, interviewSettings	Generates client UUID if needed · stores in const · injected into every WS frame sent
Coding Editor	Event-driven DOM	currentCode, language, testResults[]	Monospace textarea · Tab→4 spaces · language dropdown injects starter code · POSTs to /run-tests
Voice Controller	State machine	isListening, isSpeaking, recogActive	SpeechRecognition.onresult fills chat input · SpeechSynthesis queues Interviewer token stream
Export Handler	Async function	—	Calls /export-transcript with format · receives blob · creates objectURL · triggers download
Phase Indicator	Reactive DOM	interview_phase from WS metadata	5-step progress bar updates when session state phase changes via WS metadata frame
Analytics Summary	DOM render	all_scores, qa_log[]	End-of-session panel: avg score, highest/lowest Q, phase breakdown, top strengths/improvements
Red Flag Banner	Conditional render	red_flags[] from WS metadata	Non-intrusive banner shown when Profiler detects memorization or inconsistency · dismissible
Settings Panel	Closure	difficulty, duration, focusAreas	Session-only non-persisted settings · adaptive difficulty passes profiler signal to Interviewer

11

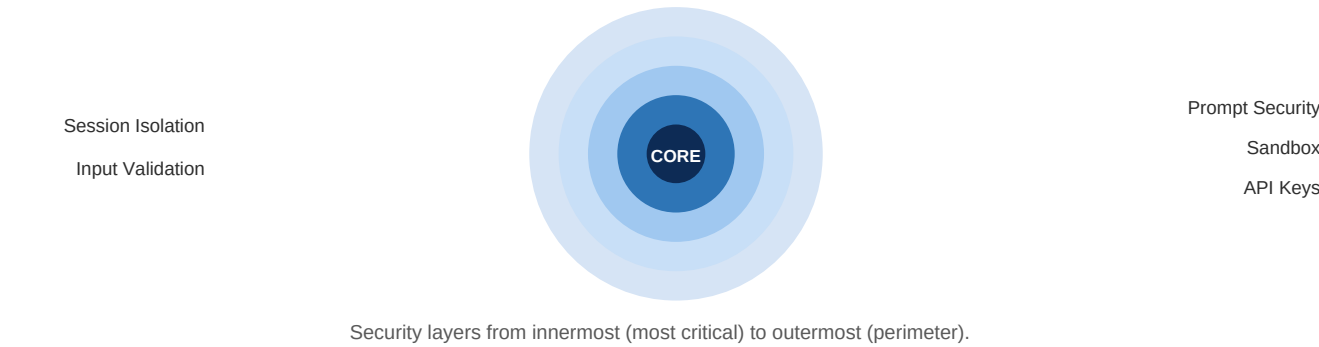
SECURITY ARCHITECTURE

The system's most distinctive security challenge is the sandboxed code execution environment for the coding round. Python's `exec()` is used with a heavily restricted whitelist-only global namespace, preventing access to system calls, file operations, or network access.

CODE EXECUTION SANDBOX — WHITELIST ARCHITECTURE

Sandbox Layer	Allowed (Whitelist)	Blocked (Everything Else)
Builtin Functions	<code>range · len · int · str · float · list · dict · tuple · set · bool · enumerate · zip · map · filter · min · max · abs · sorted · sum · round</code>	<code>open · __import__ · exec · eval · compile · globals · locals · vars · dir · getattr · setattr · delattr · chr · ord</code>
Module Imports	None — <code>__import__</code> is blocked entirely at the builtin level. No import statement can succeed inside the sandbox.	<code>os · sys · subprocess · socket · requests · shutil · pathlib · threading · multiprocessing · ctypes</code> — all <code>stdlib</code>
AST Sanitizer	Safe arithmetic · comparisons · function definitions · list/dict comprehensions · basic control flow	Import nodes · Attribute access on <code>__class__ / __bases__ / __subclasses__</code> · Exec/Eval AST nodes · restricted name Calls
Execution Scope	<code>exec(code, safe_globals, {})</code> — isolated local namespace dict. No access to application globals or Flask context.	Application state · session data · Flask request context · database connections · file system — all inaccessible
Resource Limits	Execution time capped by threading timeout wrapper. Memory monitored at OS level.	Infinite loops (caught by timeout) · large memory allocations · recursive stack overflows

SECURITY CONTROLS MATRIX



Security Control	Category	Implementation Detail & Threat Mitigated
Session UUID Isolation	Access Control	UUID4 <code>session_id</code> · Redis keys as <code>'session:{uuid}'</code> · ChromaDB collections as <code>'session_{uuid}'</code> · prevents session enumeration attacks
File Upload Validation	Input Validation	PDF MIME type only · size limit enforced · PyPDF parses in-memory · temp file deleted immediately · no user-controlled path traversal
API Key Isolation	Credentials	GROQ_API_KEY via python-dotenv only · never logged · never returned in API responses · .env excluded from version control

Security Control	Category	Implementation Detail & Threat Mitigated
Session TTL Expiry	Data Retention	Redis EXPIRE set to 21600s (6h) on creation · prevents indefinite PII retention · ChromaDB collections pruned on session end
Input Sanitization	Injection Prevent	Answer text length + charset validated · session IDs validated as UUID format · no raw SQL · Redis/ChromaDB use parameterized ops
Voice Privacy	Data Privacy	Web Speech API audio processed entirely in browser · no audio bytes transmitted to server · only transcribed text sent
LLM Prompt Isolation	Prompt Injection	User input passed as 'user' message role only · system prompts are hardcoded strings · never constructed from user input
CORS Configuration	Network Security	flask-cors restricts allowed origins in production · prevents unauthorized cross-origin API access from third-party sites
Export Path Control	File Security	Export paths computed server-side from session_id + format only · no user-controlled paths · files deleted after streaming
Error Handling	Info Exposure	Generic error messages to client · stack traces logged server-side only · no internal detail exposed in API responses

12 SCALABILITY & DEPLOYMENT

The application server is fully stateless per-request — all mutable state lives in Redis or in-memory dict. Multiple Flask instances can serve any request interchangeably, enabling horizontal scaling with a load balancer providing WebSocket affinity routing.

HORIZONTAL SCALING TOPOLOGY

Layer	Component	Scaling Strategy	Notes
1	Client Browsers	Unlimited — stateless HTTP/WS clients	WebSocket reconnect handled client-side with retry logic
2	Load Balancer	Standard round-robin for HTTP · WS affinity	Nginx upstream with proxy_pass + Upgrade/Connection headers
3	Flask Instances	Horizontal — add instances freely	Fully stateless · any instance handles any request · Docker/K8s
4	Redis	Redis Cluster mode for horizontal sharding	Sessions hash-distributed by session_id · transparent to app
5	ChromaDB	HTTP server mode · shared mounted volume	Single ChromaDB server serves N Flask instances over HTTP
6	Groq API	External stateless — unlimited concurrency	Each Flask instance makes independent Groq calls · rate-limited per key

ENVIRONMENT CONFIGURATION REFERENCE

Variable	Required	Default	Purpose & Impact
GROQ_API_KEY	★ YES	—	Authenticates all Groq API calls. App startup fails without this variable. No fallback.
REDIS_URL	No	redis://localhost/0	Full Redis connection URL. Falls back to in-memory dict silently if connection fails.
SESSION_TTL_SECONDS	No	21600 (6 hours)	Redis EXPIRE TTL. Lower = reduced PII retention. Zero disables expiry entirely.
CHROMA_DIR	No	./chroma_store	Filesystem path for ChromaDB persistence. Must be writable. Share via volume in multi-instance.
AGENT_WORKERS	No	8	ThreadPoolExecutor pool size. Increase on machines with more CPU cores for higher throughput.
FLASK_ENV	No	production	Set to 'development' for debug mode and auto-reload. Never use development in production.
PORT	No	5000	Flask server port. Override when running multiple instances or when 5000 is conflicted.

Variable	Required	Default	Purpose & Impact
MAX_UPLOAD_SIZE_MB	No	10	Maximum PDF file size for /upload-context. Larger values increase PyPDF memory usage.

CODE QUALITY ASSESSMENT

Dimension	Rating	Evidence
Separation of Concerns	★★★★★ Strong	Agent classes fully isolated from Flask routing. Each agent has one responsibility. Clean module boundaries.
Error Handling	★★★★★ Strong	Graceful Redis/ChromaDB fallbacks. Agent failures caught and surfaced as user-friendly error states.
Performance	★★★★★ Strong	Parallel agent execution. ChromaDB cache. Redis O(1) ops. Groq streaming minimizes perceived latency.
Security	★★★★★ Strong	Sandbox whitelist. Session isolation. Prompt injection prevention via message role separation.
Maintainability	★★★★■ Good	Clear boundaries and naming conventions. Type hints used. README covers config. No inline docstrings.
Test Coverage	★★■■■■ Weak	No unit, integration, or E2E tests in the repository. Agent logic untestable without Groq client mock.
Observability	★★■■■■ Weak	No structured logging, no metrics collection, no distributed tracing. Relies on Flask stderr output.
API Documentation	★★★ ■■■ Partial	README covers quickstart and config. No OpenAPI/Swagger spec. No agent docstrings.

FUTURE DEVELOPMENT ROADMAP

Priority	Enhancement	Effort	Expected Impact
P0 ★	Formal Test Suite	High	Unit tests per agent with Groq mock · integration + E2E tests · enables confident CI/CD pipeline
P0 ★	Structured Logging	Medium	JSON logs with session_id + request_id · ELK/Datadog integration · production observability
P1 ◆	Async Agent Pipeline	Medium	asyncio.gather() for Profiler + Grader in parallel · expected 40–60% per-turn latency reduction
P1 ◆	Longitudinal Tracking	High	Cross-session skill progression DB · improvement trend reports · persistent user account concept
P1 ◆	Multi-language Support	Medium	Parameterize agent system prompts for non-English interviews · multilingual embeddings in ChromaDB
P2 ●	OpenAPI Documentation	Low	Auto-generate OpenAPI 3.0 spec via flask-openapi3 · interactive Swagger UI at /docs endpoint
P2 ●	Additional Role Profiles	Low	SRE · TPM · Research Engineer · Product Designer · Growth PM · Solutions Architect profiles

Priority	Enhancement	Effort	Expected Impact
P2 ●	Admin Dashboard	Medium	Session analytics aggregator: completion rates · avg scores by role · common red flag patterns
P3 ▲	Fine-tuned Grader	Very High	Fine-tune LLaMA 7B on graded Q&A; pairs to replace Grader's Groq call — improving scoring consistency
P3 ▲	Enterprise SSO	High	SAML/OIDC for org deployments · user identity tied to session · persistent history · audit compliance

◆ **Conclusion:**

Insight demonstrates that a thoughtfully decomposed multi-agent architecture produces qualitatively superior output compared to a monolithic prompt. The technology choices compound each other: Groq's speed makes streaming natural, Redis makes Flask instances stateless and scalable, ChromaDB makes interviews resume-aware, vanilla JS keeps the frontend lean, and Web Speech API adds voice capability at zero marginal cost. The system is production-ready for its core use case with clear, prioritized extensions for enterprise-scale deployment.